

TJSC 2026 - Analista de Sistemas

Arquitetura de Sistemas e Integração

Aula 7 - Persistência com JPA, Hibernate 4.3+, Hibernate Envers e Flyway

Material autoral - padrão FGV - teoria objetiva, aprofundamento técnico e 20 questões comentadas

Recorte da aula. Esta aula cobre a Aula 7 da trilha: Persistência com JPA, Hibernate 4.3+, Hibernate Envers e Flyway. No edital, o foco direto está nos itens 1.11 e 1.12 da disciplina Arquitetura de Sistemas e Integração: persistência com JPA e Hibernate versão 4.3 ou superior; Hibernate Envers e Flyway.

Estratégia de prova. A FGV tende a cobrar distinções entre especificação e implementação, persistência de dados e migração de schema, auditoria de entidades e versionamento de banco, além de consequências arquiteturais: transação, contexto de persistência, Lazy Loading, N+1, ownership de schema por serviço e evolução segura de banco em aplicações distribuídas.

1. Identificação da aula e aderência ao edital

Item da trilha/edital	Cobertura nesta aula	Foco FGV
1.11 Persistência com JPA e Hibernate 4.3+	JPA como especificação; Hibernate como implementação/ORM; EntityManager, Session, entidade, lifecycle, consultas, transações, fetch, cache e armadilhas.	Distinguir especificação, provider, API, SQL gerado e efeitos de ciclo de vida.
1.12 Hibernate Envers e Flyway	Envers para auditoria/versionamento de entidades; Flyway para versionamento/migração de schema de banco.	Separar auditoria de dados, log de aplicação, backup e migração de schema.
Conexão com arquitetura e integração	Persistência em APIs, microsserviços, schema por serviço, deploy, CI/CD e compatibilidade de contratos.	Evitar respostas de tutorial; pensar em decisão arquitetural.

A aula não transforma persistência em curso de SQL nem em tutorial de framework. O foco é reconhecer o papel arquitetural de JPA, Hibernate, Envers e Flyway em aplicações Java/Spring/Quarkus integradas por APIs e executadas em ambientes corporativos.

2. Vocabulário essencial antes da teoria

Termo	Expansão / definição objetiva	Pegadinha FGV
JPA	Java Persistence API, hoje Jakarta Persistence. Especificação para persistência e mapeamento objeto-relacional em Java.	Dizer que JPA é uma implementação concreta como Hibernate.
ORM	Object-Relational Mapping. Técnica/ferramenta para mapear objetos Java para tabelas relacionais.	Achar que ORM elimina conhecimento de SQL, transação e modelo relacional.
Hibernate ORM	Implementação de ORM e provider JPA muito usada no ecossistema Java.	Dizer que Hibernate é a especificação JPA.
Entidade	Classe persistente com identidade, normalmente anotada com @Entity e @Id.	Confundir entidade de domínio com DTO, tabela, JSON ou record imutável.
EntityManager	Interface JPA para gerenciar entidades, contexto de persistência, consultas e ciclo de vida.	Tratar como DAO simples ou conexão JDBC.
Session	Interface nativa do Hibernate para interação com o mecanismo de persistência.	Confundir com sessão HTTP de usuário.

Persistence Context	Contexto no qual entidades gerenciadas são acompanhadas; funciona como cache de primeiro nível.	Achar que é cache distribuído ou banco temporário.
Dirty checking	Deteção automática de alterações em entidades gerenciadas para sincronização com o banco.	Achar que sempre é preciso chamar update explicitamente em entidade managed.
Flush	Sincronização do estado do contexto de persistência com o banco.	Confundir flush com commit.
Lazy/Eager	Estratégias de carregamento de associações/dados.	Dizer que EAGER sempre melhora desempenho ou que LAZY sempre é seguro.
Envers	Módulo do Hibernate para auditoria/versionamento histórico de entidades.	Confundir com Flyway, backup, log geral ou versionamento de schema.
Flyway	Ferramenta de migrations para versionar e aplicar alterações no schema do banco.	Confundir com ORM ou auditoria de dados comerciais.

3. JPA: especificação de persistência, não implementação

JPA, originalmente Java Persistence API e hoje Jakarta Persistence, **define um padrão para gerenciamento de persistência e mapeamento objeto-relacional em aplicações Java**. O ponto central para a prova é: **JPA é especificação**. Hibernate, EclipseLink e outros são implementações/provedores. **A especificação define conceitos como EntityManager, entidades, relacionamentos, JPQL e contexto de persistência**.

No edital aparece Hibernate 4.3 ou superior. Hibernate 4.3 foi associado ao período de JPA 2.1 e pacote javax.persistence; versões atuais do Jakarta Persistence usam jakarta.persistence. Em prova, a banca não deve cobrar migração de pacote, mas pode tentar confundir nomes: Java Persistence API, Jakarta Persistence, JPA e provider ORM.

Conceito	Papel correto	Não confundir com
JPA	Contrato/especificação de persistência.	Framework específico ou banco de dados.
Hibernate	Implementação ORM e provider JPA.	A própria especificação JPA.
EntityManager	API padrão para persistência e consultas.	Repository do Spring Data, JDBC Connection ou Session HTTP.
JPQL	Linguagem de consulta orientada a entidades.	SQL nativo contra tabelas.
persistence.xml / config	Configuração da unidade de persistência ou datasource, dependendo do ambiente.	Migration tool ou controle de versão de banco.

3.1 Entidade JPA e mapeamento objeto-relacional

Uma entidade JPA representa uma classe persistente. Ela possui identidade, geralmente por um atributo anotado com `@Id`. Em aplicações web, a entidade costuma estar no núcleo de persistência, mas **não deve ser automaticamente exposta como contrato de API**. Para APIs REST, é comum mapear Entity para DTO, evitando vazamento de estrutura interna, **problemas de lazy loading e acoplamento entre contrato externo e modelo relacional**.

```
@Entity
@Table(name = "processo")
public class Processo {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 30)
    private String numero;
```

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "classe_id")
private ClasseProcessual classe;
}
```

No exemplo, @Entity torna a classe persistente; @Table ajusta a tabela; @Id define identidade; @Column ajusta coluna; @ManyToOne/@JoinColumn mapeiam associação. A FGV pode tentar dizer que toda anotação de entidade define endpoint REST, o que é falso: persistência e API são camadas diferentes.

3.2 Ciclo de vida da entidade

Estado	Descrição	Exemplo de pegadinha
Transient/new	Objeto Java criado, ainda não associado ao contexto de persistência e sem persistência garantida no banco.	Achar que new Processo() já insere registro.
Managed	Entidade associada ao persistence context; alterações podem ser detectadas por dirty checking.	Achar que sempre precisa chamar update.
Detached	Entidade já existiu/tem identidade, mas não está mais gerenciada pelo contexto atual.	Achar que alterar objeto detached sempre atualiza o banco.
Removed	Entidade marcada para remoção no contexto; exclusão efetiva ocorre na sincronização/commit.	Achar que remove sempre executa DELETE imediatamente.

Operações importantes: persist torna uma nova entidade gerenciada; find busca por chave primária e retorna entidade managed; merge copia o estado de uma entidade detached para uma instância managed e retorna essa instância; remove agenda remoção; detach/clear retiram entidades do contexto; flush sincroniza alterações com o banco, mas não é sinônimo de commit.

4. Hibernate 4.3+: provider JPA e ORM na prática

Hibernate ORM é uma solução de mapeamento objeto-relacional para persistência de modelos de domínio em bancos relacionais. Ele pode ser usado por APIs JPA, como EntityManager, e também por sua API nativa, como Session. Para a FGV, o ponto não é decorar versão: é saber o que o Hibernate faz e quais erros arquiteturais decorrem do uso superficial do ORM.

Aspecto	Hibernate/JPA	Armadilha
SQL gerado	O ORM gera SQL a partir do modelo e das consultas.	Achar que o desenvolvedor nunca precisa entender SQL.
Session/EntityManager	Gerencia ciclo de vida e persistência.	Confundir com sessão de usuário no navegador.
Dialect	Adapta SQL gerado ao banco específico.	Achar que torna todo SQL nativo portátil automaticamente.
First-level cache	Persistence context por unidade de trabalho.	Confundir com cache distribuído de aplicação.
Second-level cache	Cache opcional e compartilhável conforme configuração.	Achar que sempre vem ativo e resolve N+1.
HQL/JPQL	Consultas orientadas a entidades.	Confundir com SQL diretamente sobre tabelas.

4.1 EntityManager versus Session

EntityManager é a API padronizada pela JPA. Session é a API nativa do Hibernate. Em aplicações corporativas, é comum programar contra EntityManager ou abstrações como Spring Data JPA/Panache Repository, mas compreender Session ajuda a reconhecer termos da documentação Hibernate.

Critério	EntityManager	Session
Origem	JPA/Jakarta Persistence.	Hibernate nativo.
Portabilidade	Maior, pois segue especificação.	Menor, porque acopla ao Hibernate.

Uso comum	Persistência padrão em aplicações Java enterprise/Spring/Quarkus.	Recursos específicos do Hibernate.
Pegadinha	Não é repositório mágico sem transação.	Não é sessão HTTP.

4.2 Persistence context, dirty checking, flush e commit

O persistence context mantém entidades gerenciadas e garante que, para uma mesma identidade persistente, exista uma instância gerenciada no contexto. **O dirty checking verifica alterações em entidades managed e gera SQL no momento de sincronização.** Isso explica por que, em JPA/Hibernate, alterar um atributo dentro de uma transação pode resultar em UPDATE sem chamada explícita de update.

```
@Transactional
public void alterarClasse(Long id, ClasseProcessual novaClasse) {
    Processo p = entityManager.find(Processo.class, id); // managed
    p.setClasse(novaClasse); // dirty checking detecta a mudança
    // Não há chamada explícita a update; o flush/commit sincroniza.
}
```

Operação	Efeito correto	Erro de prova
flush	Sincroniza alterações pendentes com o banco dentro da transação.	Dizer que confirma definitivamente a transação.
commit	Confirma a transação.	Confundir com simples envio de SQL.
rollback	Desfaz a transação no banco.	Achar que desfaz automaticamente efeitos externos já disparados.
merge	Copia estado do detached para uma entidade managed retornada.	Achar que o mesmo objeto detached vira managed.

4.3 Fetch, LazyInitializationException e problema N+1

Fetch define quando carregar dados associados. Em JPA, muitas associações to-one são EAGER por padrão e coleções to-many são LAZY por padrão. Porém, padrão de anotação não é plano de performance. O candidato deve saber que LAZY pode causar LazyInitializationException se acessado fora do contexto, e EAGER pode gerar consultas pesadas ou N+1.

Situação	O que acontece	Correção arquitetural típica
Acessar coleção LAZY fora da transação	Pode ocorrer LazyInitializationException no Hibernate.	Carregar o dado no caso de uso, usar DTO/projeção/fetch join/EntityGraph.
Listar processos e acessar classe de cada processo sem plano de fetch	Pode ocorrer N+1 queries.	Fetch join, batch fetching, projeção ou consulta desenhada para a tela/API.
Trocar tudo para EAGER	Pode carregar dados demais e piorar performance.	Definir fetch conforme caso de uso.
Expor entidade diretamente no JSON	Serializador pode disparar lazy loading e vaziar modelo interno.	Usar DTOs e montar resposta dentro da fronteira transacional.

Regra de prova: **N+1 não é resolvido simplesmente trocando tudo para EAGER.** A solução correta depende do caso de uso: consulta com fetch join, EntityGraph, projeção, paginação, batch size ou desenho de endpoint mais adequado.

4.4 Cascades, orphanRemoval e associações

Cascade propaga operações de persistência entre entidades associadas: PERSIST, MERGE, REMOVE, REFRESH, DETACH ou ALL. Isso não é a mesma coisa que ON DELETE CASCADE do banco. **orphanRemoval remove o filho quando ele deixa de pertencer à associação**, sendo comum em composição forte.

Recurso	Uso correto	Pegadinha
cascade = PERSIST	Persistir filho ao persistir agregado/pai.	Achar que cria relacionamento no banco sem @JoinColumn/@JoinTable.
cascade = REMOVE	Remover filhos quando remover pai, se fizer sentido no domínio.	Usar em associações compartilhadas e apagar dado indevido.
orphanRemoval = true	Remover filho órfão da	Confundir com apenas quebrar FK sem

	coleção/associação.	excluir registro.
mappedBy	Indica o lado inverso de uma associação bidirecional.	Achar que ambos os lados são donos da FK.

5. Persistência em APIs, microsserviços e integração

Em arquitetura de sistemas, JPA/Hibernate não devem ser estudados como simples forma de “gravar objeto no banco”. Eles influenciam fronteira transacional, acoplamento, contrato de API, escalabilidade e evolução do schema. Em um backend Java/Quarkus ou Spring, o erro clássico é deixar a entidade JPA atravessar todas as camadas: banco -> entidade -> JSON -> Angular. Isso funciona no protótipo e vira problema em produção.

Decisão	Boa prática arquitetural	Risco FGV
Contrato REST	Expor DTOs/representações de API, não entidades JPA diretamente.	Confundir entidade persistente com payload público.
Transação	Abrir transação no caso de uso/serviço de aplicação.	Fazer lazy loading na camada de apresentação.
Microsserviço	Cada serviço tende a possuir seu próprio modelo e schema ou ao menos fronteira clara de dados.	Defender banco compartilhado como integração universal.
Consulta	Projetar consulta conforme endpoint, paginação e dados necessários.	Achar que ORM escolhe sozinho o plano ótimo.
Evolução de schema	Usar migrations versionadas e compatíveis com deploy.	Confiar em hibernate.hbm2ddl.auto=update em produção.

Em microsserviços, compartilhar banco entre serviços reduz autonomia e aumenta acoplamento. A integração deve ocorrer por APIs, eventos ou mecanismos controlados. O banco pode existir em tecnologia relacional, mas a fronteira de ownership importa: quem é dono do schema, quem pode migrá-lo e quem depende de suas tabelas.

6. Hibernate Envers: auditoria e versionamento de entidades

Hibernate Envers é um módulo do Hibernate ORM para auditoria/versionamento de entidades persistentes. A ideia central é simples: ao anotar uma entidade com `@Audited`, o Envers registra alterações em tabelas de auditoria, permitindo consultar versões históricas da entidade.

```
@Entity
@Audited
public class Processo {
    @Id
    private Long id;
    private String numero;
    private String status;
}
```

Elemento	Significado	Pegadinha
@Audited	Marca entidade ou atributo para auditoria pelo Envers.	Achar que audita automaticamente qualquer tabela do banco.
Tabela *_AUD	Tabela de histórico da entidade auditada.	Confundir com backup completo do banco.
Revision	Número/registro de revisão associado a um conjunto de alterações.	Confundir com versão de migration Flyway.
REVTYPE	Tipo de alteração, como inclusão, alteração ou exclusão.	Confundir com status negocial do processo.
AuditReader	API para consultar histórico auditado.	Confundir com EntityManager comum para estado atual.

Envers é útil quando a aplicação precisa saber “como estava esta entidade em determinada revisão”. Ele não substitui log de segurança, trilha completa de requisições, backup, replicação, CDC, auditoria de acesso ou

versionamento de schema. Em regra, ele captura alterações feitas pelo ciclo de vida do Hibernate/JPA; alterações diretas no banco por script ou sistema externo podem não aparecer no histórico Envers.

A FGV pode explorar o termo “versionamento”. **Envers versiona/audita dados de entidades**. Flyway versiona **mudanças estruturais de banco**. Git versiona **código-fonte**. Backup preserva **cópias recuperáveis**. São coisas próximas na linguagem comum, mas diferentes em arquitetura.

7. Flyway: migrations e versionamento de schema

Flyway é ferramenta de migrations de banco de dados. Ela aplica scripts versionados/repetíveis para levar o schema de um estado a outro de forma rastreável. Em aplicações Spring Boot, por exemplo, a presença do flyway-core pode permitir que migrations em classpath:db/migration sejam executadas na inicialização. O ponto de prova: **Flyway controla a evolução do schema, não é ORM e não audita alterações de entidades**.

Tipo / comando	Função	Pegadinha FGV
Versioned migration	Script aplicado uma única vez em ordem de versão, como V1__criar_tabela.sql.	Alterar script já aplicado sem consequências.
Repeatable migration	Script sem versão, como R__view_processos.sql, reaplicado quando checksum muda.	Achar que substitui versioned migration para tudo.
flyway_schema_history	Tabela de metadados das migrations aplicadas.	Confundir com tabela de auditoria negocial.
migrate	Aplica migrations pendentes.	Confundir com rollback.
validate	Valida scripts contra histórico e checksums.	Achar que corrige automaticamente inconsistências.
repair	Repara metadados em situações controladas.	Achar que desfaz script ruim ou recupera dados.
baseline	Cria ponto de partida para banco existente.	Achar que apaga tudo e recria do zero.
clean	Remove objetos do schema conforme configuração; perigoso em produção.	Tratar como comando comum e seguro em qualquer ambiente.

```
-- src/main/resources/db/migration/V1__criar_tabela_processo.sql
CREATE TABLE processo (
    id BIGINT PRIMARY KEY,
    numero VARCHAR(30) NOT NULL,
    status VARCHAR(30) NOT NULL
);

-- src/main/resources/db/migration/V2__adicionar_coluna_classe.sql
ALTER TABLE processo ADD COLUMN classe_id BIGINT;
```

Regra forte: **migration aplicada não deve ser editada como se fosse arquivo comum**. Se a equipe altera um script já aplicado, o checksum pode divergir do histórico e o validate tende a acusar problema. Em ambiente corporativo, cria-se nova migration para evoluir o schema.

Ferramenta	Versiona o quê?	Não faz
Flyway	Schema e scripts de migração de banco.	Não audita alterações de dados de entidades.
Envers	Histórico de mudanças em entidades auditadas.	Não cria/evolui schema de banco como migration.
Hibernate/JPA	Mapeia entidades e executa persistência.	Não substitui governança de migrations.
Git	Código-fonte e artefatos textuais versionados.	Não garante que o banco real foi migrado.
Backup	Cópia recuperável de dados/ambiente.	Não é estratégia de migration ou auditoria de entidade.

8. Evolução segura de schema em sistemas distribuídos

Em ambiente distribuído e escalável, várias versões de aplicação podem coexistir durante deploy. Por isso, migrations devem ser compatíveis com a versão antiga e a nova, sempre que houver rolling update, blue/green ou canary. A FGV pode cobrar a consequência arquitetural: não basta escrever ALTER TABLE; é preciso preservar compatibilidade de contrato e dados durante a transição.

Etapa	Exemplo	Por que importa
Expandir	Adicionar coluna nova nullable ou tabela nova sem quebrar código antigo.	Permite rodar versão antiga e nova.
Adaptar aplicação	Nova versão passa a preencher/ler a nova estrutura.	Migração de comportamento com compatibilidade.
Backfill	Popular dados antigos em lote controlado.	Evita inconsistência histórica.
Contrair	Remover coluna antiga só quando nenhuma versão depender dela.	Evita quebrar instâncias antigas durante deploy.

Ponto de prova: Flyway é excelente para controlar scripts, mas não decide sozinho se uma migration é segura para deploy sem parada. Essa é uma decisão arquitetural de compatibilidade entre banco, aplicação e consumidores.

8.1 Mapeamento relacional: cardinalidade, dono da relação e tabela de junção

A FGV pode cobrar JPA/Hibernate de forma conceitual, sem pedir código completo. O caminho mais provável é apresentar um cenário de modelagem e perguntar qual anotação ou consequência é adequada. O candidato precisa separar cardinalidade do domínio, ownership do relacionamento e implementação física no banco.

Relação JPA	Ideia relacional típica	Ponto de prova
@ManyToOne	Muitos registros da entidade atual apontam para um registro da entidade alvo por FK.	Muitos processos pertencem a uma classe processual; normalmente é lado dono com @JoinColumn.
@OneToMany	Um registro se relaciona com muitos registros da outra tabela; geralmente lado inverso usa mappedBy.	Não basta criar lista no Java; é preciso entender quem possui a FK.
@OneToOne	Associação um para um, com FK única ou PK compartilhada conforme modelagem.	Não confundir com simples DTO embutido.
@ManyToMany	Geralmente exige tabela de junção.	Pode ser inadequado quando a associação possui atributos próprios; nesse caso vira entidade associativa.
@Embedded / @Embeddable	Valor composto armazenado na tabela da entidade proprietária.	Não possui identidade própria como entidade.

Exemplo de pegadinha: se uma associação processo-documento possui atributos próprios, como data de juntada, usuário de juntada ou sigilo específico naquela relação, tratá-la como simples @ManyToMany pode empobrecer o modelo. O mais aderente costuma ser criar uma entidade de associação, como ProcessoDocumento.

```
@Entity
public class ProcessoDocumento {
    @EmbeddedId
    private ProcessoDocumentoId id;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("processoId")
    private Processo processo;

    @ManyToOne(fetch = FetchType.LAZY)
    @MapsId("documentoId")
    private Documento documento;
```



```
private LocalDateTime dataJuntada;
}
```

Não é necessário decorar `@EmbeddedId` ou `@MapsId` para o TJSC, mas é útil entender o raciocínio: quando a relação possui atributos próprios, ela deixa de ser apenas ligação e passa a ser elemento relevante do domínio.

8.2 Consultas: JPQL, HQL, Criteria, native SQL e projeções

JPA define JPQL, uma linguagem de consulta orientada a entidades. Hibernate possui HQL, muito próximo e historicamente mais amplo. Também é possível executar SQL nativo, mas isso reduz portabilidade e aproxima o código do dialeto do banco. Em prova, a FGV pode misturar entidade, tabela, atributo Java e coluna SQL.

Consulta	Opera sobre	Quando faz sentido	Pegadinha
JPQL	Entidades e atributos Java.	Consultas portáteis orientadas ao modelo JPA.	Escrever nome de tabela/coluna como se fosse SQL.
HQL	Entidades Hibernate.	Recursos do Hibernate e sintaxe orientada a objetos.	Tratar como SQL nativo obrigatório.
Criteria API	Modelo programático/tipado de consulta.	Consultas dinâmicas e montagem segura por código.	Achar que é ferramenta de migration.
Native SQL	Tabelas e colunas reais do banco.	Otimizações específicas, procedures, funções/dialetos.	Achar que continua totalmente portátil.
Projection/DTO	Subconjunto de campos necessário ao caso de uso.	Listagens, relatórios, endpoints enxutos.	Carregar entidade completa para toda tela simples.

```
// JPQL: usa entidade e atributo Java, não tabela e coluna física.
```

```
SELECT p FROM Processo p WHERE p.numero = :numero
```

```
// Projeção: evita carregar o agregado inteiro quando a tela precisa de poucos dados.
```

```
SELECT new br.jus.tjsc.api.ProcessoResumoDTO(p.id, p.numero, p.status)
```

```
FROM Processo p
```

```
WHERE p.status = :status
```

Ponto FGV: consulta eficiente em API não é necessariamente “buscar entidade inteira e deixar o JSON filtrar”. Em sistemas com dados sensíveis e volumes altos, projeções e DTOs ajudam a reduzir acoplamento, tráfego e risco de vazamento de atributos internos.

8.3 Transações, concorrência e locking

Persistência em arquitetura também envolve consistência. Em JPA/Hibernate, **o limite transacional define quando alterações são vistas, sincronizadas e confirmadas.** Em sistemas web, um caso de uso de escrita normalmente precisa de transação. Para concorrência, JPA permite estratégias como optimistic locking, frequentemente com `@Version`, e pessimistic locking, apoiado pelo banco.

Tema	Ideia central	Pegadinha FGV
<code>@Transactional</code>	Delimita unidade de trabalho transacional em frameworks como Spring/Quarkus.	Achar que toda chamada HTTP é transacional por natureza.
Optimistic locking	Detecta conflito por versão; útil quando conflitos são possíveis, mas não constantes.	Achar que bloqueia linha fisicamente o tempo todo.
<code>@Version</code>	Campo de versão usado pelo provider para detectar atualização concorrente.	Confundir com versão Flyway ou revisão Envers.
Pessimistic locking	Bloqueio no banco para impedir concorrência sobre recurso.	Usar indiscriminadamente e reduzir escalabilidade.
Long transaction	Transação longa pode reter recursos e aumentar contenção.	Achar que manter transação aberta durante chamada externa é sempre correto.

```
@Entity
```

```
public class Minuta {
```



```

@Id
private Long id;

@Version
private Long versao;

private String texto;
}

```

Regra de prova: @Version de JPA/Hibernate não é Envers e não é Flyway. @Version apoia controle de concorrência otimista. Envers registra histórico de alterações. Flyway controla scripts de evolução do schema.

8.4 Cache, paginação e batch processing

Hibernate mantém cache de primeiro nível no persistence context. Isso evita que a mesma entidade, na mesma unidade de trabalho, seja materializada diversas vezes. Porém, esse cache cresce conforme entidades managed são carregadas. Em processamento em lote, é comum usar flush e clear periódicos para evitar excesso de memória e reduzir custo de dirty checking.

Recurso	Função	Limite
First-level cache	Cache obrigatório do persistence context.	Vive no contexto; não é cache distribuído entre instâncias.
Second-level cache	Cache opcional compartilhado entre contextos, se configurado.	Não vem como solução universal e exige invalidação/coerência.
Paginação	Evita carregar volumes grandes em memória.	Paginar com fetch join em coleções exige cuidado.
Batch insert/update	Processa grandes volumes com flush/clear por blocos.	Carregar tudo como managed pode estourar memória.

```

for (int i = 0; i < itens.size(); i++) {
    entityManager.persist(itens.get(i));
    if (i % 50 == 0) {
        entityManager.flush();
        entityManager.clear();
    }
}

```

Para a FGV, isso pode aparecer como leitura de cenário: “a aplicação carrega 500 mil entidades managed no mesmo EntityManager”. A resposta correta não é culpar apenas o banco; o persistence context também consome memória e precisa de fronteira adequada.

8.5 Envers em maior detalhe: REVINFO, REVTYPE e limitações

Em uma configuração comum, Envers cria tabelas de auditoria para entidades auditadas e mantém uma tabela de revisões. Cada alteração auditada fica associada a uma revisão. A tabela de auditoria costuma ter o identificador original da entidade, a revisão e um tipo de revisão, como inclusão, alteração ou exclusão.

Elemento típico	Papel	Atenção
REVINFO ou tabela de revisão	Registra metadados da revisão.	Pode ser customizada por RevisionEntity/Listener.
REV	Número da revisão.	Não é versão de migration Flyway.
REVTYPE	Tipo de operação: inclusão, alteração, exclusão.	Não é status negocial da entidade.
*_AUD	Tabela histórica da entidade auditada.	Não substitui tabela principal nem backup.
@NotAudited	Permite excluir atributo/associação da auditoria.	Não remove o campo da entidade principal.

Limitações importantes: Envers não registra automaticamente mudanças feitas fora do Hibernate; não resolve auditoria de leitura/ acesso; não substitui trilha de segurança; não é estratégia de recuperação de desastre; e não deve ser usado como desculpa para ignorar logs técnicos e controles de acesso.

8.6 Flyway em pipeline: ordem, histórico e falhas

Em pipeline corporativa, Flyway costuma aparecer antes da aplicação começar a usar o schema novo. A ferramenta compara scripts disponíveis com o histórico do banco, aplica pendentes e falha se encontrar divergências relevantes. Essa falha é desejável: impede que ambientes diferentes continuem divergindo silenciosamente.

Cenário	Leitura correta	Erro comum
Banco novo	Migrations V1, V2, V3 são aplicadas em ordem.	Aplicar manualmente só a última migration.
Banco existente sem histórico	baseline pode marcar ponto inicial de controle.	Confundir baseline com apagar e recriar tudo.
Script aplicado foi editado	validate detecta checksum incompatível.	Editar migration aplicada para "consertar rapidinho".
Falha parcial	Analisar histórico, transação do banco e comando repair com cautela.	Usar repair como rollback mágico.
Ambientes múltiplos	Migrations devem ser reproduzíveis e versionadas.	Alterar script conforme ambiente sem controle.

Em bancos diferentes, DDL transacional pode variar. Por isso, uma migration precisa ser testada no SGBD alvo. Flyway organiza a execução, mas não transforma automaticamente uma alteração perigosa em alteração segura.

8.7 Caso aplicado ao TJSC: API, entidade, Envers e Flyway no mesmo fluxo

Imagine um módulo interno de consulta e movimentação processual. A aplicação Angular chama uma API Java. A API não devolve a entidade Processo diretamente; devolve ProcessoDTO. O serviço de aplicação abre transação, carrega a entidade Processo, altera o status e registra o histórico da entidade via Envers. Antes do deploy, uma migration Flyway adicionou a coluna nova necessária ao status detalhado. Esse desenho separa responsabilidades.

Peça	Responsabilidade no caso	Não deve assumir
Angular	Consumir contrato REST/JSON.	Conhecer lazy loading ou tabelas internas.
DTO	Representar dados do contrato de API.	Ser entidade managed.
Entity JPA	Representar estado persistente interno.	Virar contrato público automaticamente.
Hibernate	Mapear e persistir entidade.	Decidir sozinho arquitetura de microserviços.
Envers	Registrar histórico das mudanças da entidade auditada.	Criar coluna nova no banco.
Flyway	Aplicar a migration que evolui o schema.	Auditar quem alterou cada entidade.

Essa separação é exatamente o tipo de raciocínio que rende ponto em Arquitetura de Sistemas e Integração: cada componente com seu papel, sem transformar ferramenta em solução universal.

9. Comparações que a FGV pode explorar

Comparação	Distinção correta
JPA x Hibernate	JPA é especificação; Hibernate é implementação/provedor ORM.
EntityManager x Repository	EntityManager é API de persistência; Repository é abstração de acesso usada em camadas de aplicação/frameworks.
JPQL/HQL x SQL	JPQL/HQL consultam entidades; SQL consulta tabelas/colunas.
Flush x Commit	Flush sincroniza; commit confirma a transação.
Managed x Detached	Managed é acompanhado pelo persistence context; detached não é atualizado automaticamente.
LAZY x EAGER	Estratégias de carregamento; nenhuma é solução universal de performance.
Envers x Flyway	Envers audita dados de entidade; Flyway migra schema.

hbm2ddl.auto x Flyway	Geração automática de schema pode ajudar em dev/test; Flyway dá controle versionado para ambientes reais.
Cascade x FK cascade	Cascade JPA propaga operações de entidade; FK cascade é regra do banco.
DTO x Entity	DTO é contrato/representação; Entity é modelo persistente.

10. Pegadinhas FGV de alto rendimento

1. JPA não é Hibernate: JPA é especificação; Hibernate é implementação/provedor.
2. Hibernate não dispensa modelagem relacional: o ORM reduz SQL manual, mas não elimina transação, índices, cardinalidade e plano de consulta.
3. EntityManager não é conexão JDBC: ele gerencia entidades e persistence context.
4. Objeto managed alterado pode gerar UPDATE por dirty checking sem chamada explícita a update.
5. merge não torna necessariamente o mesmo objeto detached gerenciado; ele retorna uma instância managed com o estado copiado.
6. flush não é commit: flush sincroniza alterações; commit confirma transação.
7. LAZY pode gerar LazyInitializationException; EAGER pode gerar carga excessiva ou N+1. Nenhum é solução mágica.
8. N+1 é problema de desenho de consulta/fetch, não apenas de banco lento.
9. Cascade REMOVE em associação compartilhada pode apagar dados indevidamente.
10. orphanRemoval remove órfão; não é apenas desvincular registro.
11. Expor entidade JPA diretamente em JSON acopla API ao modelo interno e pode disparar lazy loading.
12. Envers audita entidades; não é Flyway, backup ou log completo de segurança.
13. Flyway versiona schema/migrations; não audita alterações negociais de registros.
14. Editar migration já aplicada tende a causar divergência de checksum; o correto é criar nova migration.
15. clean é perigoso e não deve ser tratado como operação comum em produção.
16. baseline serve para iniciar controle sobre banco existente; não é recriação automática do schema.
17. Repeatable migration é útil para views/procedures; versioned migration continua essencial para evolução incremental.
18. hbm2ddl.auto=update em produção não substitui governança de migration.
19. Em microserviços, banco compartilhado como integração universal reduz autonomia e aumenta acoplamento.
20. Flyway aplica scripts, mas não garante por si só deploy zero downtime; a migration precisa ser compatível com versões em execução.

11. Questões autorais - padrão TJSC/FGV

Resolva as questões antes de consultar o gabarito. As questões são autorais e foram calibradas para o estilo de discriminação conceitual da FGV; não reproduzem itens reais de prova.

Questão 1. Em uma aplicação Java corporativa, uma equipe decide usar JPA com Hibernate para persistir entidades em banco relacional. Considerando a relação entre JPA e Hibernate, assinale a afirmativa mais adequada.

- A) JPA é uma implementação concreta de ORM fornecida pelo Hibernate.
- B) Hibernate é uma especificação abstrata que exige um provider JPA para funcionar.
- C) JPA e Hibernate são sinônimos, diferenciando-se apenas pelo pacote Java utilizado.
- D) JPA define uma especificação de persistência e Hibernate pode atuar como implementação/provedor dessa especificação.
- E) JPA substitui banco relacional por armazenamento em memória gerenciado pela JVM.

Questão 2. Considere um método transacional que busca uma entidade Processo pelo EntityManager, altera seu status e retorna sem chamar update explicitamente. Em JPA/Hibernate, a atualização pode ocorrer porque:

- A) toda entidade Java é persistida automaticamente pelo garbage collector.
- B) entidades managed são acompanhadas pelo persistence context, que pode detectar alterações por dirty checking.
- C) o JSON de resposta sempre sincroniza o objeto com o banco.
- D) o Flyway executa automaticamente UPDATES em entidades alteradas.
- E) o Envers substitui o EntityManager no ciclo de vida transacional.

Questão 3. Uma entidade Documento foi carregada dentro de uma transação. Depois do fechamento do EntityManager, a aplicação altera um atributo desse mesmo objeto em memória. Sem nova transação ou merge, essa alteração:

- A) será persistida automaticamente, pois a entidade tem @Id.
- B) será enviada ao banco pelo próximo comando Flyway migrate.
- C) será auditada pelo Envers e por isso atualizada no banco.
- D) será sincronizada no próximo GET HTTP.
- E) não será persistida automaticamente, pois o objeto está detached em relação ao contexto encerrado.

Questão 4. Em JPA, a operação merge aplicada a uma entidade detached:

- A) transforma obrigatoriamente o próprio objeto detached em managed, sem retorno relevante.
- B) executa sempre um INSERT imediato, ainda que exista registro com a mesma identidade.
- C) copia o estado do objeto detached para uma instância managed e retorna essa instância gerenciada.
- D) é equivalente ao comando Flyway repair.
- E) remove a entidade do banco se houver divergência de versão.

Questão 5. Sobre flush e commit em JPA/Hibernate, assinale a alternativa correta.

- A) Flush sincroniza o estado do contexto de persistência com o banco, mas não se confunde com confirmação definitiva da transação.
- B) Flush é sinônimo de commit e sempre torna a transação irreversível.
- C) Commit apenas limpa o cache de primeiro nível, sem confirmar transação.
- D) Flush cria migrations Flyway pendentes.
- E) Commit é uma operação exclusiva do Envers.

Questão 6. Uma API REST expõe diretamente entidades JPA como JSON. Em determinado endpoint, a serialização acessa uma coleção LAZY após o fechamento da transação, provocando erro em runtime. O problema descrito é compatível com:

- A) baseline do Flyway.
- B) schema history inconsistente.
- C) conflito entre REV e REVTYPE no Envers.
- D) LazyInitializationException ou falha de inicialização tardia fora do contexto de persistência.
- E) ausência de @GeneratedValue em migrations repeatable.

Questão 7. O problema conhecido como N+1 em JPA/Hibernate ocorre, tipicamente, quando:

- A) o Flyway executa uma migration V1 e depois uma migration V2.
- B) uma consulta inicial carrega N entidades e o acesso a associação de cada uma dispara consultas adicionais.
- C) o Envers cria uma tabela de auditoria para cada entidade anotada.
- D) o EntityManager executa exatamente uma consulta com fetch join.
- E) o banco possui uma chave primária composta por N colunas mais uma coluna.

Questão 8. Sobre estratégias de fetch em JPA/Hibernate, assinale a afirmativa mais adequada.

- A) EAGER deve ser usado em todas as associações para evitar qualquer problema de performance.
- B) LAZY sempre garante ausência de consultas adicionais.
- C) Fetch é irrelevante em aplicações REST, pois o JSON decide sozinho o que carregar.
- D) Trocar todas as associações para EAGER é a solução correta para N+1.
- E) LAZY e EAGER definem estratégias de carregamento; o plano adequado depende do caso de uso, transação e consulta.

Questão 9. Em uma associação bidirecional JPA, o atributo mappedBy indica:

- A) o nome da tabela de auditoria do Envers.
- B) o script Flyway responsável por criar a FK.
- C) o lado inverso da associação, apontando para o atributo que é dono do relacionamento.
- D) o tipo de fetch obrigatório da associação.
- E) o endpoint REST que expõe o relacionamento.

Questão 10. A respeito de cascade e orphanRemoval em JPA, assinale a alternativa correta.

- A) cascade REMOVE deve ser aplicado indiscriminadamente a toda associação ManyToMany.
- B) orphanRemoval pode remover o filho que deixa de pertencer à associação, quando isso representa composição/posse no domínio.
- C) cascade PERSIST substitui a necessidade de mapeamento de @JoinColumn ou @JoinTable.
- D) cascade e ON DELETE CASCADE são sempre a mesma regra executada no mesmo lugar.
- E) orphanRemoval apenas coloca a FK como nula e nunca remove registro.

Questão 11. Hibernate Envers é mais adequadamente descrito como:

- A) módulo de auditoria/versionamento de entidades persistentes, capaz de manter histórico de alterações em tabelas de auditoria.
- B) ferramenta de migração de schema que aplica scripts V1__ e V2__.
- C) especificação oficial que substitui JPA.
- D) servidor de mensageria usado para eventos comerciais.
- E) mecanismo de load balancing entre instâncias de serviços.

Questão 12. Um sistema precisa consultar como determinada entidade Processo estava em uma revisão histórica. Se essa entidade estiver corretamente auditada, o recurso mais aderente é:

- A) Flyway clean.
- B) Spring Cloud Gateway.
- C) EntityManagerFactory.close().
- D) Hibernate Envers, por meio de suas tabelas e APIs de auditoria.
- E) hbm2ddl.auto=update.

Questão 13. Sobre Envers, assinale a alternativa incorreta.

- A) Pode auditar entidades anotadas com @Audited.
- B) Pode criar tabelas de auditoria associadas às entidades auditadas.
- C) Substitui backups, logs de segurança, auditoria de acesso e migrations de schema em uma única solução.
- D) Permite consultar versões históricas de entidades auditadas.
- E) Distingue alterações por revisões e tipo de operação.

Questão 14. Flyway, no contexto de arquitetura de sistemas, é usado principalmente para:

- A) mapear objetos Java para tabelas relacionais.
- B) auditar alterações históricas de atributos de entidades.

- C) transformar EntityManager em API REST.
- D) criar automaticamente DTOs a partir de entidades.
- E) controlar e aplicar migrations de schema de banco de dados de forma versionada.

Questão 15. Um arquivo Flyway denominado V2__adicionar_coluna_classe.sql foi aplicado em homologação. Depois disso, um desenvolvedor alterou o conteúdo desse mesmo arquivo. Em regra, o risco mais direto é:

- A) o Hibernate trocar automaticamente LAZY por EAGER.
- B) divergência de checksum em validação, pois migration versionada aplicada não deve ser alterada livremente.
- C) o Envers remover a tabela _AUD correspondente.
- D) o EntityManager transformar todas as entidades em detached.
- E) o Gateway API deixar de rotear chamadas REST.

Questão 16. Sobre repeatable migrations no Flyway, assinale a alternativa correta.

- A) São úteis para objetos reaplicáveis, como views ou procedures, e podem ser reaplicadas quando seu checksum muda.
- B) São scripts versionados obrigatoriamente no formato V1__nome.sql.
- C) Substituem toda migration versionada em qualquer cenário.
- D) São tabelas de auditoria geradas pelo Envers.
- E) São endpoints REST repetidos em múltiplos microsserviços.

Questão 17. Uma equipe usa hibernate.hbm2ddl.auto=update em produção como única estratégia de evolução de banco. Do ponto de vista arquitetural, a decisão é problemática porque:

- A) Hibernate não consegue mapear entidades para tabelas relacionais.
- B) JPA proíbe qualquer alteração de schema.
- C) Envers deixa de funcionar quando há EntityManager.
- D) geração automática de schema não substitui controle versionado, revisável e previsível de migrations como Flyway.
- E) Flyway só funciona em bancos NoSQL.

Questão 18. Em microsserviços, a decisão de vários serviços acessarem diretamente as mesmas tabelas de um banco compartilhado tende a:

- A) aumentar autonomia, pois cada serviço poderá alterar livremente o schema.
- B) eliminar a necessidade de contratos de API.
- C) aumentar acoplamento e dificultar evolução independente dos serviços e de suas migrations.
- D) transformar automaticamente o banco em message broker.
- E) substituir a necessidade de transações.

Questão 19. Considere a seguinte sequência de deploy para uma mudança de banco: adicionar coluna nova sem remover a antiga, adaptar aplicação para preencher a coluna, executar backfill, e só depois remover a coluna antiga. Esse padrão está associado a:

- A) uso de entidade detached como cache distribuído.
- B) cascade REMOVE em associação compartilhada.
- C) substituição de JPA por JDBC puro.
- D) rollback automático do Envers.
- E) estratégia expand/contract para evolução compatível de schema em ambientes distribuídos.

Questão 20. Em uma questão da FGV, aparece a afirmação: "Flyway e Envers são equivalentes, pois ambos versionam o banco de dados". A avaliação correta é:

- A) correta, pois ambos armazenam migrations na mesma tabela _AUD.

B) incorreta: Flyway versiona/aplica mudanças de schema, enquanto Envers audita/versiona alterações de entidades persistentes.

C) correta apenas se a aplicação usar EntityManager.

D) incorreta apenas porque Envers depende de RabbitMQ.

E) correta, desde que as entidades tenham @Id.

12. Gabarito resumido

Questão	Gabarito	Questão	Gabarito	Questão	Gabarito	Questão	Gabarito
1	D	6	D	11	A	16	A
2	B	7	B	12	D	17	D
3	E	8	E	13	C	18	C
4	C	9	C	14	E	19	E
5	A	10	B	15	B	20	B

Gabarito em sequência: 1D - 2B - 3E - 4C - 5A - 6D - 7B - 8E - 9C - 10B - 11A - 12D - 13C - 14E - 15B - 16A - 17D - 18C - 19E - 20B

13. Comentários completos

Questão 1 - Gabarito: D. D é correta: JPA é especificação e Hibernate pode ser provider/implementação. A inverte os papéis. B inverte novamente: Hibernate não é a especificação. C apaga uma distinção que a FGV adora. E inventa substituição do banco relacional por memória JVM.

Questão 2 - Gabarito: B. B é correta: uma entidade managed é acompanhada pelo persistence context e alterações podem ser detectadas por dirty checking. A é absurda, mas tenta explorar automatismo. C confunde serialização com persistência. D atribui a Flyway papel de ORM. E atribui a Envers função de EntityManager.

Questão 3 - Gabarito: E. E é correta: após fechamento do contexto, o objeto fica detached; mudanças nele não são sincronizadas automaticamente. A erra ao achar que @Id basta. B confunde migration com atualização de dados. C exagera o Envers. D confunde HTTP com persistência.

Questão 4 - Gabarito: C. C é correta: merge copia estado do objeto detached para uma instância managed e retorna essa instância. A é a pegadinha clássica: o objeto passado não necessariamente vira managed. B erra ao dizer INSERT sempre. D confunde com Flyway. E inventa efeito de remoção por divergência.

Questão 5 - Gabarito: A. A é correta: flush sincroniza alterações, mas commit confirma a transação. B confunde conceitos. C reduz commit a limpeza de cache. D coloca Flyway onde há persistência. E atribui commit ao Envers.

Questão 6 - Gabarito: D. D é correta: acesso tardio a associação lazy fora do contexto pode gerar LazyInitializationException. A, B e E pertencem a Flyway/migrations. C mistura Envers com problema de carregamento.

Questão 7 - Gabarito: B. B descreve N+1: uma consulta inicial seguida de consultas adicionais por associação. A é fluxo normal de migrations. C descreve efeito de auditoria, não N+1. D é justamente uma possível solução. E cria confusão com chave composta.

Questão 8 - Gabarito: E. E é correta: fetch depende do caso de uso, transação e consulta. A e D são generalizações perigosas. B erra porque LAZY pode gerar consultas adicionais ou erro fora do contexto. C atribui ao JSON uma decisão de persistência.

Questão 9 - Gabarito: C. C é correta: mappedBy indica o lado inverso, apontando para o atributo dono do relacionamento. A confunde com Envers. B confunde com Flyway. D confunde associação com estratégia de fetch. E mistura persistência com API REST.

Questão 10 - Gabarito: B. B é correta: orphanRemoval remove o filho quando ele deixa de pertencer à associação, se isso representa a regra de domínio. A é perigosa em ManyToMany. C erra porque cascade não cria mapeamento. D confunde cascade ORM com FK cascade. E nega o efeito de remoção.

Questão 11 - Gabarito: A. A é correta: Envers audita/versiona entidades em tabelas de auditoria. B descreve Flyway. C confunde com especificação JPA. D descreve broker. E descreve balanceamento.

Questão 12 - Gabarito: D. D é correta: Envers permite consultar histórico de entidades auditadas. A é comando destrutivo do Flyway. B é gateway. C fecha fábrica de EntityManager. E é geração automática de schema, não auditoria histórica.

Questão 13 - Gabarito: C. A incorreta é C: Envers não substitui backup, log de segurança, auditoria de acesso e migrations. A, B, D e E descrevem usos adequados do Envers. A pegadinha está em transformar auditoria de entidade em solução universal.

Questão 14 - Gabarito: E. E é correta: Flyway controla e aplica migrations de schema. A é ORM/Hibernate. B é Envers. C e D misturam persistência com API/DTO. Flyway executa scripts de banco, não mapeia entidades.

Questão 15 - Gabarito: B. B é correta: alterar migration já aplicada causa risco de checksum divergente e falha de validate. A, C, D e E deslocam o tema para fetch, Envers, lifecycle e gateway, sem relação direta.

Questão 16 - Gabarito: A. A é correta: repeatable migrations são úteis para objetos reaplicáveis e são reaplicadas quando mudam. B descreve versioned migration. C exagera. D descreve Envers. E confunde migrations com endpoints.

Questão 17 - Gabarito: D. D é correta: hbm2ddl.auto=update não substitui migration versionada governada. A é falsa, pois Hibernate mapeia ORM. B inventa proibição. C não procede. E restringe Flyway indevidamente.

Questão 18 - Gabarito: C. C é correta: banco compartilhado entre serviços aumenta acoplamento e dificulta evolução independente. A inverte o efeito. B parece tentadora, mas elimina contratos só na aparência. D confunde banco com broker. E ignora transações.

Questão 19 - Gabarito: E. E é correta: sequência expand/contract preserva compatibilidade em deploy distribuído. A trata detached como cache. B fala de cascade. C não tem relação. D atribui rollback automático ao Envers.

Questão 20 - Gabarito: B. B é correta: Flyway versiona/aplica mudanças de schema; Envers audita/versiona mudanças em entidades. A inventa tabela comum. C condiciona indevidamente. D mistura com mensageria. E reduz equivalência à presença de @Id.

14. Checklist final de cobertura da aula

- Diferenciar JPA de Hibernate e entender provider/implementação ORM.
- Reconhecer EntityManager, Session, persistence context, lifecycle e dirty checking.
- Distinguir managed, detached, transient e removed.
- Separar flush de commit e merge de persist/update direto.
- Identificar riscos de LAZY/EAGER, LazyInitializationException e N+1.
- Usar DTOs e fronteiras transacionais em APIs, evitando expor entidades como contrato externo.
- Entender Envers como auditoria/versionamento de entidades, não como migration, backup ou log universal.
- Entender Flyway como versionamento/migração de schema, com migrations versioned/repeatable, checksum e schema history.
- Perceber a relação entre persistência e arquitetura distribuída: ownership de schema, deploy compatível e migrações seguras.

15. Referências técnicas consultadas

- Edital TJSC n. 10/2026, retificado em 10/04/2026 - disciplina Arquitetura de Sistemas e Integração, Analista de Sistemas.
- Manual Mestre de Calibração FGV para Aulas e Questões - Projeto TJSC 2026.
- Jakarta Persistence Specification 3.1 - <https://jakarta.ee/specifications/persistence/3.1/jakarta-persistence-spec-3.1.html>
- Oracle Java EE EntityManager API - <https://docs.oracle.com/javaee/7/api/javax/persistence/EntityManager.html>

- Hibernate ORM User Guide - <https://docs.jboss.org/hibernate/orm/>
- Hibernate ORM 4.x Developer Guide - https://docs.jboss.org/hibernate/orm/4.2/devguide/en-US/html_single/
- Hibernate Envers Documentation - <https://hibernate.org/orm/envers/> e <https://docs.jboss.org/envers/docs/>
- Redgate Flyway Documentation - Migrations - <https://documentation.red-gate.com/fd/migrations-271585107.html>
- Redgate Flyway Documentation - Repeatable Migrations - <https://documentation.red-gate.com/fd/repeatable-migrations-273973335.html>
- Spring Boot Database Initialization/Flyway - <https://docs.spring.io/spring-boot/docs/2.0.0.M5/reference/html/howto-database-initialization.html>